

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ»

КАФЕДРА ТЕОРЕТИЧЕСКОЙ И ПРИКЛАДНОЙ ИНФОРМАТИКИ

Лабораторная работа № 5
по дисциплине «Теория информации и криптография»
на тему «Криптографические библиотеки»

Факультет: ФПМИ

Группа: ПМИ-32

Студенты: Весёлый Д., Ворончук И., Лыкова М.

Бригада №2

Преподаватели: Авдеенко Т. В., Кутузова И. А.

НОВОСИБИРСК 2025

Цель работы: Познакомиться с существующими криптографическими библиотеками. Научиться использовать сторонние криптографические библиотеки при разработке собственных приложений.

Задание:

- I. Найти криптографическую библиотеку.
- II. Реализовать приложение с графическим интерфейсом, позволяющее выполнять следующие действия.
 1. Шифровать и дешифровать выбранный пользователем файл с использованием одного или нескольких симметричных алгоритмов шифрования:
 - 1) результаты шифрования и дешифрования должны сохраняться в файлы;
 - 2) требуемые параметры шифрования, такие как ключ, вектор инициализации и пр., должны считываться из файла.
 2. Шифровать и дешифровать выбранный пользователем файл с использованием одного асимметричного алгоритма шифрования:
 - 1) реализовать процедуру генерации пары открытый–закрытый ключ;
 - 2) результаты шифрования и дешифрования должны сохраняться в файлы;
 - 3) требуемые параметры шифрования должны считываться из файла.
 3. Вычислять и проверять электронную цифровую подпись для выбранного пользователем файла с использованием одного из алгоритмов:
 - 1) результаты вычисления подписи должны сохраняться в файлы;
 - 2) требуемые параметры вычисления и проверки подписи должны считываться из файла.
 4. Вычислять хэш-значения для выбранного пользователем файла по одному или нескольким алгоритмам хэширования, при этом вычисленное хэш-значение должно сохраняться в файл.
- III. Протестировать правильность работы реализованного приложения.

Описание метода решения заданий.

Разрабатывается приложение с графическим интерфейсом, предоставляющее пользователю единый инструмент для выполнения основных криптографических операций: шифрование и дешифрование симметричными алгоритмами, шифрование и дешифрование асимметричными алгоритмами, вычисление и проверку электронной цифровой подписи (ЭЦП), хэширование.

Решение построено на принципах модульности и разделения ответственности: каждая криптографическая функция инкапсулирована в отдельный логический модуль (вкладку интерфейса), вспомогательные операции (работа с файлами, логирование, диалоги) вынесены в утилитарные функции.

Симметричное шифрование

Принцип работы: Один и тот же ключ используется для шифрования и дешифрования данных.

Реализованные алгоритмы: AES (размер ключа: 256 бит, размер блока: 128 бит), DES (размер ключа: 56 бит, размер блока: 64 бит), 3DES (размер ключа: 168 бит, размер блока: 64 бит).

В каждом используется режим шифрования CBC, каждый блок открытого текста перед шифрованием комбинируется с предыдущим блоком шифротекста с помощью операции XOR. Требуется заполнения (padding) для последнего блока.

Асимметричное шифрование

Принцип работы: Используется пара ключей - открытый (для шифрования) и закрытый (для дешифрования).

Параметры реализации: Размер ключа: 256 или 512 байт;

RSA имеет ограничение на размер шифруемых данных (меньше размера модуля ключа), применяется двухуровневый подход: сначала генерируется случайный симметричный ключ, затем данные шифруются симметричным алгоритмом. Симметричный ключ шифруется асимметричным алгоритмом. В выходной файл записываются: зашифрованный ключ, вектор инициализации и зашифрованные данные.

Электронная цифровая подпись:

Принцип работы: Подпись создается путём шифрования хэш-значения данных закрытым ключом. Проверка осуществляется расшифрованием подписи открытым ключом и сравнением с вычисленным хэшем.

Хэширование

Принцип работы: Хэш-функция преобразует произвольные данные в фиксированную битовую строку с свойствами детерминированности (одинаковый вход → одинаковый выход), необратимости (невозможно восстановить вход по хэшу), лавинного эффект (минимальное изменение входа радикально меняет выход) и стойкости к коллизиям (сложно найти два разных входа с одинаковым хэшем).

Описание разработанного программного средства.

Была выбрана криптографическая библиотека cryptography - библиотека для языка Python, предоставляющая высокоуровневые криптографические примитивы для безопасной обработки данных. Библиотека реализует все требуемые операции, низкоуровневый интерфейс hazmat позволяет наглядно демонстрировать принципы работы криптографических примитивов, режимов шифрования и схем заполнения.

Импортируемые модули:

```
from cryptography.hazmat.primitives.ciphers import Cipher,
algorithms, modes
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import rsa, padding
as rsa_padding
from cryptography.hazmat.primitives.padding import PKCS7
from cryptography.hazmat.backends import default_backend
```

Разработанное программное средство представляет собой настольное приложение с графическим интерфейсом, предназначенное для выполнения комплекса криптографических операций над файлами произвольного формата и объёма. Программа реализует фундаментальные механизмы защиты информации: симметричное и асимметричное шифрование, создание и верификацию электронной цифровой подписи, а также вычисление хэш-сумм.

Программное средство разделено на четыре функциональных модуля, каждый из которых отвечает за отдельный класс криптографических операций.

Модуль симметричного шифрования (`class SymmetricTab`) поддерживает алгоритмы AES, DES и 3DES в режимах CBC и CFB, генерация криптографически стойких ключей и векторов инициализации происходит автоматически функцией `os.urandom()`.

`cipher.encryptor()` (метод `encrypt()` в коде) производит инициализацию процесса шифрования.

`enc.update(data) + enc.finalize()` преобразуют открытый текст в шифротекст.

`cipher.decryptor()` инициализирует процесс дешифрования.

`dec.update(data) + dec.finalize()` преобразовывает шифротекст в открытый текст.

`symmetric.generate_params(algo)` производит генерацию ключа и IV.

Модуль асимметричного шифрования (`class AsymmetricTab`) генерирует случайный ключ `aes_key` длиной 64 и 32 байт, реализует гибридные схемы шифрования (сначала происходит шифрование данных симметрично с помощью AES-CBC с `aes_key` и `iv`, затем происходит асимметричное шифрование `aes_key` открытым RSA-ключом `public_key` (RSA)). Получаем `{enc_key, iv, ciphertext}`. Дешифрование `aes_key` происходит закрытым RSA-ключом асимметрично с помощью `private_key` (RSA). Расшифровка данных происходит симметрично через восстановленные `aes_key + iv`.

Пример генерации пары ключей RSA:

```
def generate_keys(self):
    size = int(self.key_size_var.get()) # 2048 или 4096 бит
    priv_key = rsa.generate_private_key(
        public_exponent=65537, # Стандартное значение
        key_size=size,
        backend=default_backend()
    )
```

`rsa.generate_private_key()` генерирует пары ключей RSA.

`pub_key.encrypt()` шифрует сеансовый AES-ключ открытым ключом RSA.
`priv_key.decrypt(enc_aes_key, rsa_padding.OAEP(...))` расшифровывает сеансового AES-ключа закрытым ключом RSA.

Модуль электронной цифровой подписи (`class SignatureTab`) формирует подпись на базе алгоритма RSA-PSS (Probabilistic Signature Scheme, делает процесс подписания вероятностным, одно и то же сообщение, подписанное дважды одним и тем же закрытым ключом, даст разные подписи) с хэш-функцией SHA-256, поддерживает вероятностную схему подписи с использованием случайного параметра для повышения криптостойкости. `salt_length=PSS.MAX_LENGTH` делает максимально возможной длину случайных данных, что делает подпись криптографически наиболее стойкой.

Подписание данных происходит через функцию `sign`:

```
def sign(private_key, data: bytes) -> bytes:
    return private_key.sign(data, _PSS(), hashes.SHA256())
```

На вход подаётся закрытый (приватный) ключ и данные (`data`), которые нужно подписать. Данные хэшируются (SHA-256), К хэшу применяется

паddинг PSS, выполняется математическая операция с закрытым ключом. На выходе получается бинарная строка - цифровая подпись.

Проверка подписи:

```
def verify(public_key, data: bytes, signature: bytes) -> bool:
    try:
        public_key.verify(signature, data, _PSS(),
hashes.SHA256())
        return True
    except InvalidSignature:
        return False
```

На вход подаются открытый (публичный) ключ автора, исходные данные (data) и полученная подпись (signature). Код заново хэширует data, пытается математически открыть подпись с помощью публичного ключа и сравнивает полученный результат с хэшем данных. Если есть совпадение, то возвращает True (подпись верна, данные не менялись). Если нет, то ловит исключение InvalidSignature и возвращает False.

Модуль хэширования (class HashTab) вычисляет хеш-сумму по алгоритмам MD5, SHA-1, SHA-256, SHA-512, SHA3-256, SHA3-512, происходит параллельное вычисление хэш-значений несколькими алгоритмами для сравнительного анализа. Используется стандартный модуль python hashlib, который вычисляет хэш-сумму файла в 16-м виде. Пользователь передаёт строку, код читает его как bytes, вызывает compute_all(ALGORITHMS, data), для каждого алгоритма hashlib создаёт объект-хэшер, прогоняет байты через внутренний буфер и финализирует вычисление и результат преобразуется в hex-строку

Пример вычисления нескольких хэшей для одного файла:

```
def compute(self):
    data = open(path, "rb").read() # Чтение файла в бинарном
    режиме
    self._last_results = {}
    lines = [f"Файл: {path}\n"]

    for algo in selected: # selected список выбранных
    пользователем алгоритмов
        h = self._compute_hash(algo, data)
        self._last_results[algo] = h
```

Тестирование программы.

Тест 1

Симметричное шифрование

1.1. Генерация параметров AES-CBC:

Ожидаемый результат:

Поля Ключ (Key, Base64) и Вектор (IV, Base64) заполнены непустыми значениями. Длина декодированного ключа: 32 байта (256 бит). Длина декодированного IV: 16 байт (128 бит)

Вывод программы:

► Параметры — ключ и IV

JSON-файл параметров: Обзор...

— или Base64 вручную —

Ключ (Key, Base64):

Вектор (IV, Base64):

1.2. Шифрование и дешифрование текста через AES-CBC

Входные данные: Алгоритм AES-CBC, текст: "Секретное сообщение", ключ и вектор инициализации из прошлого теста.

Ожидаемый результат: в окне вывода появится Base64-строка. После шифрования она автоматически подставится в поле для дешифровки.

Вывод программы:

Алгоритм: AES-CBC

► Параметры — ключ и IV

JSON-файл параметров: Обзор...

— или Base64 вручную —

Ключ (Key, Base64):

Вектор (IV, Base64):

► Шифрование — открытые данные

Текст (открытый):

— или файл —

Входной файл: Обзор...

Выходной файл (.enc): Сохранить...

► Дешифрование — данные шифтекста

Шифртекст (Base64):

— или файл —

Файл шифртекста (.enc/.bin): Обзор...

Выходной файл (.dec): Сохранить...

Расшифрованный текст:

Сохранить в .t

Генерировать параметры Зашифровать Дешифровать

Журнал операций

[OK] Зашифровано (AES-CBC)
Шифртекст (Base64):
iceXu56jE/exCvuWTi8RO9UVJv8+9+QL6oki+eXlBppyUiajVnT8RYQGo7
Шифртекст вставлен в поле дешифрования

[OK] Дешифровано (AES-CBC) — текст отображен в поле результата

1.3. Шифрование и дешифрование файла через AES-CFB

Входные данные: алгоритм AES-CFB,

Ожидаемый результат: зашифрованный текст и оригинальный совпадает побайтово.

Вывод программы:

The screenshot shows a web application interface for AES-CFB encryption and decryption. The interface is divided into several sections:

- Алгоритм:** A dropdown menu set to "AES-CFB".
- Параметры — ключ и IV:** A section for setting parameters. It includes a "JSON-файл параметров:" field with an "Обзор..." button, a "Ключ (Key, Base64):" field with the value "n4tEDOqo/a9z5Z9VTnqpwOGEWtFkJcJOLMIACFIYoww=", and a "Вектор (IV, Base64):" field with the value "KNMeterHOK4DSTOTHK1L+aQ==".
- Шифрование — открытые данные:** A section for encrypting text. It includes a "Текст (открытый):" field with the value "Aboba", a "Входной файл:" field with an "Обзор..." button, and a "Выходной файл (.enc):" field with a "Сохранить..." button.
- Дешифрование — данные шифртекста:** A section for decrypting ciphertext. It includes a "Шифртекст (Base64):" field with the value "nIObegM=", a "Файл шифртекста (.enc/.bin):" field with an "Обзор..." button, and a "Выходной файл (.dec):" field with a "Сохранить..." button.
- Расшифрованный текст:** A field showing the decrypted text "Aboba".
- Buttons:** "Генерировать параметры", "Зашифровать", "Дешифровать", "Очистить", and "Сохранить вывод в .txt".
- Журнал операций:** A log showing the following messages:
 - [OK] Параметры для AES-CFB
Key : n4tEDOqo/a9z5Z9VTnqpwOGEWtFkJcJOLMIACFIYoww=
IV : KNTMeterHOK4DSTOTHK1L+aQ==
 - [OK] Зашифровано (AES-CFB)
Шифртекст (Base64):
nIObegM=
 - Шифртекст вставлен в поле дешифрования
 - [OK] Дешифровано (AES-CFB) — текст отображен в поле результата

1.4. Шифрование DES-CBC

Входные данные: алгоритм DES-CBC, ключ размером 8 байт после декодирования, IV 8 байт. Текст DES test.

Ожидаемый результат: длина декодированного ключа равна 8 байт.

Вывод программы:

Симм. шифрование | Асимм. шифрование | Цифровая подпись | Хэширование

Алгоритм: DES-CBC

► Параметры — ключ и IV

JSON-файл параметров: Обзор...

— или Base64 вручную —

Ключ (Key, Base64):

Вектор (IV, Base64):

► Шифрование — открытые данные

Текст (открытый):

— или файл —

Входной файл: Обзор...

Выходной файл (.enc): Сохранить...

► Дешифрование — данные шифртекста

Шифртекст (Base64):

— или файл —

Файл шифртекста (.enc/.bin): Обзор...

Выходной файл (.dec): Сохранить...

Расшифрованный текст:

Сохранить в .txt

Генерировать параметры Зашифровать Дешифровать

Журнал операций

```
[OK] Параметры для DES-CBC
Key : ZctK0duFHEc=
IV : z0wbl8XkRMo=
[OK] Зашифровано (DES-CBC)
Шифртекст (Base64):
hFQDjg1OgUG4Pq06X39+8g==
Шифртекст вставлен в поле дешифрования
[OK] Дешифровано (DES-CBC) — текст отображен в поле результата
```

1.6. Генерация параметров 3DES-CBC

Входные данные: алгоритм 3DES-CBC.

Ожидаемый результат: длина декодированного ключа: 24 байта (168 бит),
длина декодированного IV: 8 байт.

Вывод программы:

Симм. шифрование | Асимм. шифрование | Цифровая подпись | Хэширование

Алгоритм: 3DES-CBC

► Параметры — ключ и IV

JSON-файл параметров: Обзор...

— или Base64 вручную —

Ключ (Key, Base64):

Вектор (IV, Base64):

► Шифрование — открытые данные

Текст (открытый):

— или файл —

Входной файл: Обзор...

Выходной файл (.enc): Сохранить...

► Дешифрование — данные шифртекста

Шифртекст (Base64):

— или файл —

Файл шифртекста (.enc/.bin): Обзор...

Выходной файл (.dec): Сохранить...

Расшифрованный текст:

Сохранить в .txt

Генерировать параметры Зашифровать Дешифровать

Журнал операций

```
[OK] Параметры для 3DES-CBC
Key : QLu6ZLX//bpo0kuFjaL8c3BqA73jK1O6
IV : XO5Ta2355xM=
[OK] Зашифровано (3DES-CBC)
Шифртекст (Base64):
VltpFXazmN0Rr2ftQ/ejOw==
Шифртекст вставлен в поле дешифрования
[OK] Дешифровано (3DES-CBC) — текст отображен в поле результата
```

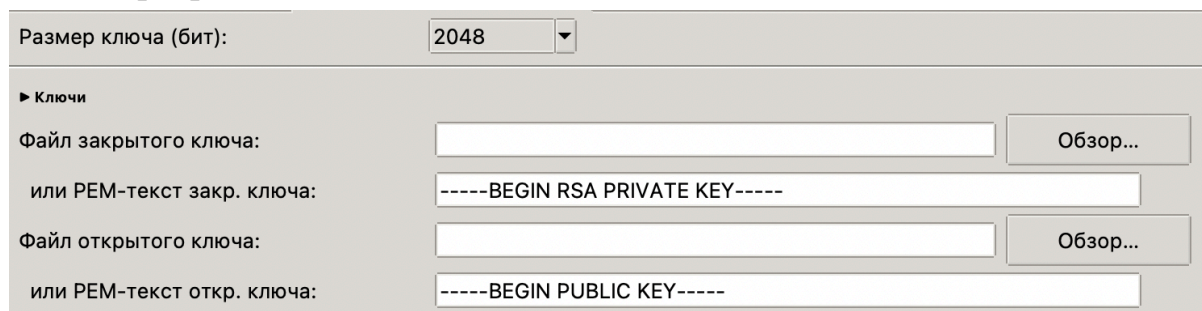
2. Асимметричное шифрование

2.1. Генерация ключевой пары RSA-2048

Входные данные: размер ключа 2048.

Ожидаемый результат: Поля закрытого ключа и открытого ключа заполнены

Вывод программы:



Ключи сгенерированы

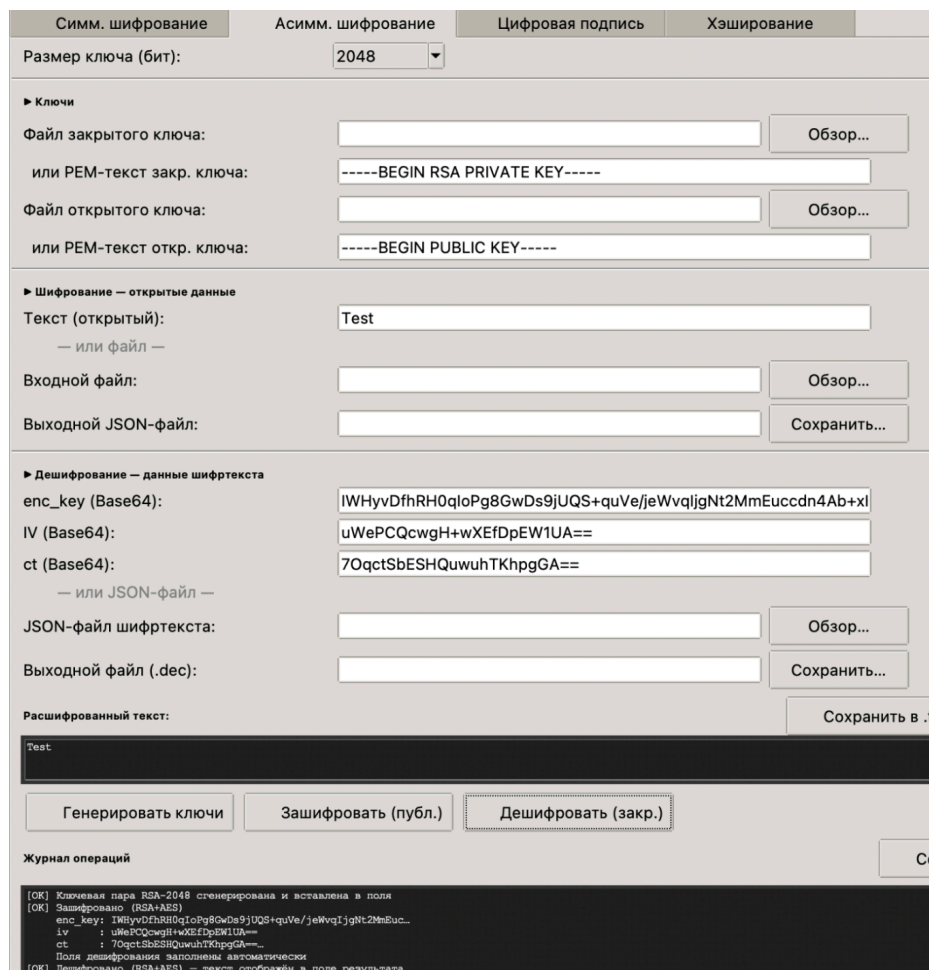
Тест 2.2. Шифрование и дешифрование текста открытым ключом

Входные данные: сгенерированные ранее ключи, текст "Test".

Ожидаемый результат: в окно вывода появляется структура

"enc_key":..., "iv":..., "ct":...; Дешифрованный текст является исходным.

Результат программы:

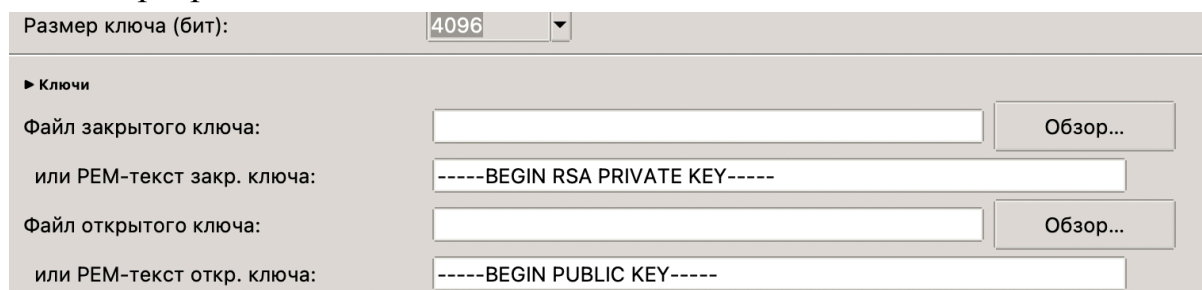


Тест 2.4. Генерация ключевой пары RSA-4096:

Входные данные: размер ключа 4096.

Ожидаемый результат: ключи сгенерированы.

Вывод программы:

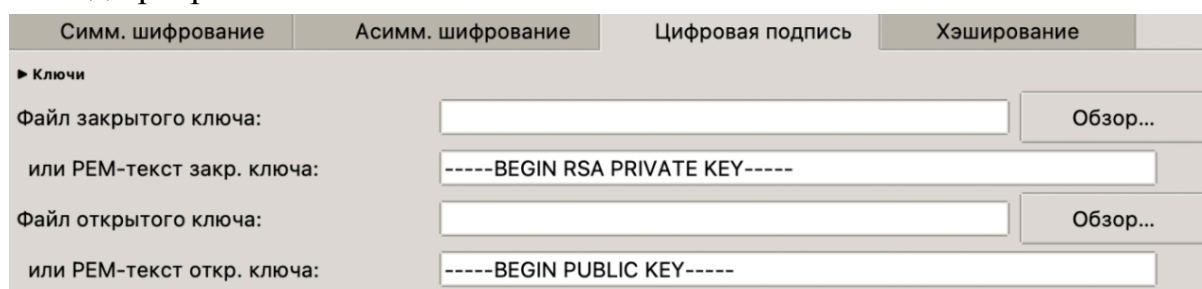


Тест 3: цифровая подпись

Тест 3.1. Генерация ключей RSA-2048 на вкладке подписи

Ожидаемый вывод: Поля “Файл закрытого ключа” и “Файл открытого ключа” заполнены.

Вывод программы:



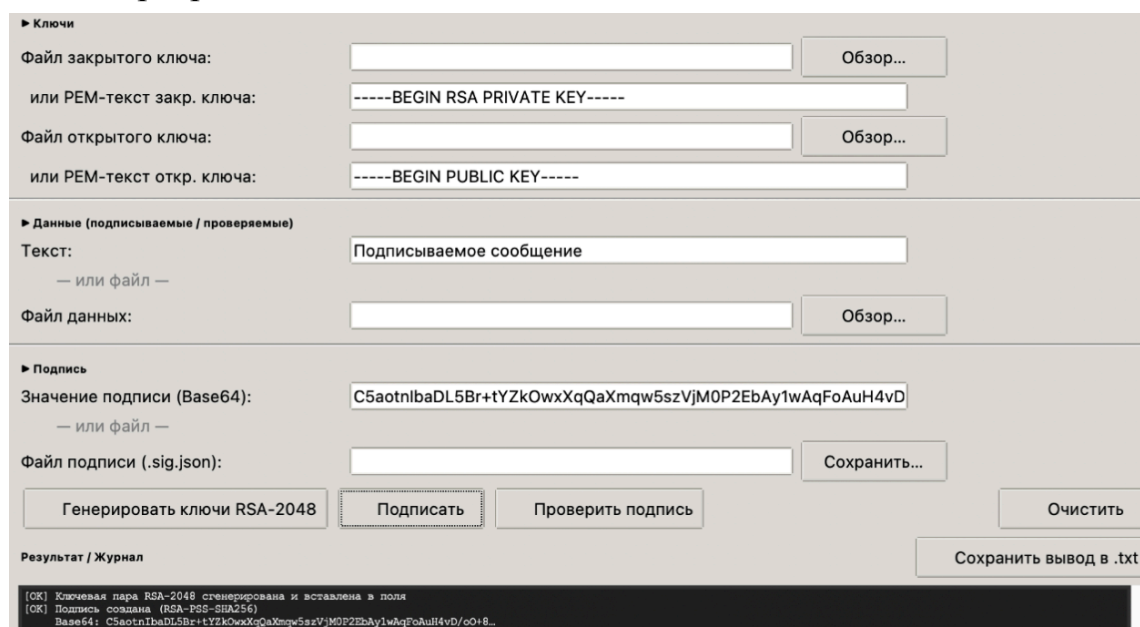
Ключи сгенерировались.

Тест 3.2. Подпись текста

Входные данные: текст сообщения - “Подписываемое сообщение”.

Ожидаемый результат: поле Base64-значение подписи заполнено (длинная Base64-строка).

Вывод программы:



Тест 3.3. Проверка действительной подписи

Входные данные: текст тот же, что и в предыдущем тесте.

Ожидаемый результат: сообщение “Подпись действительна!”

Вывод программы:

Симм. шифрование | Асимм. шифрование | **Цифровая подпись** | Хэширование

► Ключи

Файл закрытого ключа: Обзор...

или PEM-текст закр. ключа:

Файл открытого ключа: Обзор...

или PEM-текст откр. ключа:

► Данные (подписываемые / проверяемые)

Текст:

— или файл —

Файл данных: Обзор...

► Подпись

Значение подписи (Base64):

— или файл —

Файл подписи (.sig.json): Сохранить...

Генерировать ключи RSA-2048 Подписать Проверить подпись Очистить

Результат / Журнал Сохранить вывод в .txt

```
[OK] Ключевая пара RSA-2048 сгенерирована и вставлена в поля
[OK] Подпись создана (RSA-PSS-SHA256)
Base64: C5aotnlbaDL5Br+tYZkOwxXqQaXmqw5szVjM0P2EbAy1wAqFoAuH4vD/co+8...
[OK] Подпись ДЕЙСТВИТЕЛЬНА
```

Тест 3.4. Проверка подписи при изменённых данных

Входные данные: изменённый текст сообщения или изменённая подпись.

Ожидаемый результат: сообщение “Подпись недействительна!”.

Вывод программы:

Симм. шифрование | Асимм. шифрование | **Цифровая подпись** | Хэширование

► Ключи

Файл закрытого ключа: Обзор...

или PEM-текст закр. ключа:

Файл открытого ключа: Обзор...

или PEM-текст откр. ключа:

► Данные (подписываемые / проверяемые)

Текст:

— или файл —

Файл данных: Обзор...

► Подпись

Значение подписи (Base64):

— или файл —

Файл подписи (.sig.json): Сохранить...

Генерировать ключи RSA-2048 Подписать Проверить подпись Очистить

Результат / Журнал Сохранить вывод в .txt

```
[OK] Ключевая пара RSA-2048 сгенерирована и вставлена в поля
[OK] Подпись создана (RSA-PSS-SHA256)
Base64: C5aotnlbaDL5Br+tYZkOwxXqQaXmqw5szVjM0P2EbAy1wAqFoAuH4vD/co+8...
[OK] Подпись ДЕЙСТВИТЕЛЬНА
[OK] Подпись НЕДЕЙСТВИТЕЛЬНА
```

Тест 4: Хэширование

4.1. Хэширование строки алгоритмом SHA-256

Входные данные: строка “Hello, World!”. Алгоритм SHA-256.

Ожидаемый результат: SHA-256:

dffd6021bb2bd5b0af676290809ec3a53191dd81c7f70a4b28688a362182986d

Вывод программы:

Входные данные

Строка для хэширования:

— или файл —

Файл:

Обзор...

Алгоритмы

☐ MD5 ☐ SHA-1 ☒ SHA-256 ☐ SHA-512 ☐ SHA3-256 ☐ SHA3-512

Вычислить хэши

Результаты хэширования

```
==== Хэширование ====
Дата/время : 2026-04-18 17:21:22
Источник    : "Hello, World!"
SHA-256     : dffd6021bb2bd5b0af676290809ec3a53191dd81c7f70a4b28688a362182986f
```

4.2. Хэширование строки сразу несколькими алгоритмами

Входные данные: строка “test”, алгоритмы MD5, SHA-1, SHA-256, SHA-512, SHA3-256, SHA3-512.

Ожидаемый результат:

MD5: 098f6bcd4621d373cade4e832627b4f6

SHA-1: a94a8fe5ccb19ba61c4c0873d391e987982fbdd3

SHA-256: 9f86d081884c7d659a2feaa0c55ad015a3bf4f1b2b0...

SHA-512: ee26b0dd4af7e749aa1a8ee3c10ae9923f618980772e473f8819...

SHA3-256: 36f028580bb02cc8272a9a020f4200e346e276ae66...

SHA3-512: 9ece086e9bac491fac5c1d1046ca11d737b92a2b2eb...

Вывод программы:

Входные данные

Строка для хэширования:

— или файл —

Файл:

Обзор...

Алгоритмы

☒ MD5 ☒ SHA-1 ☒ SHA-256 ☒ SHA-512 ☒ SHA3-256 ☒ SHA3-512

Вычислить хэши

Результаты хэширования

```
==== Хэширование ====
Дата/время : 2026-04-18 17:23:11
Источник    : "test"
MD5         : 098f6bcd4621d373cade4e832627b4f6
SHA-1       : a94a8fe5ccb19ba61c4c0873d391e987982fbdd3
SHA-256     : 9f86d081884c7d659a2feaa0c55ad015a3bf4f1b2b05b0f00a08
SHA-512     : ee26b0dd4af7e749aa1a8ee3c10ae9923f618980772e473f8819a5d4940e0db27ac185f8a0e1d5f84f88bc887fd67b143732c304cc5fa9ad8e6f57f50028a8ff
SHA3-256    : 36f028580bb02cc8272a9a020f4200e346e276ae664e45ee80745574e2f5ab80
SHA3-512    : 9ece086e9bac491fac5c1d1046ca11d737b92a2b2ebd93f005d7b710110c0a678288166e7f7be796883a4f2e9b3ca9f484f521d0ce464345cc1aec96779149c14
```


Вывод: В рамках работы на языке Python с использованием библиотеки cryptography было разработано настольное приложение, реализующее 13 алгоритмов шифрования, хеширования и электронной подписи. Корректность функций подтверждена полным покрытием ручного тестирования и автоматизированными тестами. В результате были освоены практические навыки работы с высокоуровневыми и низкоуровневыми криптографическими интерфейсами, а также принципы организации модульного тестирования.

Приложение

Код программы.

[symmetric.py](#)

```
# Симметричное шифрование: AES-CBC, AES-CFB, DES-CBC, 3DES-CBC
```

```
import os
```

```
import base64
```

```
from cryptography.hazmat.primitives.ciphers import Cipher,  
algorithms, modes
```

```
from cryptography.hazmat.primitives.padding import PKCS7
```

```
from cryptography.hazmat.backends import default_backend
```

```
ALGORITHMS = ["AES-CBC", "AES-CFB", "DES-CBC", "3DES-CBC"]
```

```
def key_iv_size(algo: str) -> tuple[int, int]:
```

```
# Возвращает (размер_ключа, размер_IV) в байтах
```

```
    match algo:
```

```
        case "AES-CBC" | "AES-CFB":
```

```
            return 32, 16          # AES-256
```

```
        case "DES-CBC":
```

```
            return 8, 8
```

```
        case "3DES-CBC":
```

```
            return 24, 8
```

```
        case _:
```

```
            raise ValueError(f"Неизвестный алгоритм: {algo}")
```

```
def block_size(algo: str) -> int:
```

```
# Возвращает размер блока в битах (для PKCS7).
```

```
    return 128 if algo.startswith("AES") else 64
```

```
def generate_params(algo: str) -> dict:
```

```
# Генерирует случайные ключ и IV для заданного алгоритма.
```

```
    key_len, iv_len = key_iv_size(algo)
```

```
    return {
```

```
        "algorithm": algo,
```

```
        "key": base64.b64encode(os.urandom(key_len)).decode(),
```

```
        "iv": base64.b64encode(os.urandom(iv_len)).decode(),
```

```
    }
```

```
def make_cipher(algo: str, key: bytes, iv: bytes):
```

```
# Создаёт объект Cipher для заданного алгоритма.
```

```
    match algo:
```

```
        case "AES-CBC":
```



```

        return Cipher(algorithms.AES(key), modes.CBC(iv),
backend=default_backend())
    case "AES-CFB":
        return Cipher(algorithms.AES(key), modes.CFB(iv),
backend=default_backend())
    case "DES-CBC":
        k = key * 3 if len(key) == 8 else key
        return Cipher(algorithms.TripleDES(k), modes.CBC(iv),
backend=default_backend())
    case "3DES-CBC":
        return Cipher(algorithms.TripleDES(key),
modes.CBC(iv), backend=default_backend())
    case _:
        raise ValueError(f"Неизвестный алгоритм: {algo}")

```

```

def encrypt(algo: str, key: bytes, iv: bytes, data: bytes) ->
bytes:

```

Шифрует данные. Для CBC-режимов применяет PKCS7 padding.

```

    if algo in ("AES-CBC", "DES-CBC", "3DES-CBC"):
        padder = PKCS7(block_size(algo)).padder()
        data = padder.update(data) + padder.finalize()

```

```

    cipher = make_cipher(algo, key, iv)
    enc = cipher.encryptor()
    return enc.update(data) + enc.finalize()

```

```

def decrypt(algo: str, key: bytes, iv: bytes, ct: bytes) -> bytes:

```

Дешифрует данные. Для CBC-режимов снимает PKCS7 padding

```

    cipher = make_cipher(algo, key, iv)
    dec = cipher.decryptor()
    pt = dec.update(ct) + dec.finalize()

```

```

    if algo in ("AES-CBC", "DES-CBC", "3DES-CBC"):
        unpadder = PKCS7(block_size(algo)).unpadder()
        pt = unpadder.update(pt) + unpadder.finalize()
    return pt

```

[asymmetric.py](#)

Асимметричное шифрование: RSA-2048 / RSA-4096 (гибридная схема RSA+AES)

```

import os

```

```

import base64

from cryptography.hazmat.primitives.ciphers import Cipher,
algorithms, modes
from cryptography.hazmat.primitives.padding import PKCS7
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import rsa, padding
as rsa_padding
from cryptography.hazmat.backends import default_backend

def generate_key_pair(key_size: int = 2048):
    # Генерирует пару RSA-ключей. Возвращает (private_key, public_key)
    private_key = rsa.generate_private_key(
        public_exponent=65537,
        key_size=key_size,
        backend=default_backend()
    )
    return private_key, private_key.public_key()

def private_key_to_pem(private_key) -> bytes:
    """Сериализует закрытый ключ в PEM (без пароля)."""
    return private_key.private_bytes(
        serialization.Encoding.PEM,
        serialization.PrivateFormat.TraditionalOpenSSL,
        serialization.NoEncryption()
    )

def public_key_to_pem(public_key) -> bytes:
    """Сериализует открытый ключ в PEM."""
    return public_key.public_bytes(
        serialization.Encoding.PEM,
        serialization.PublicFormat.SubjectPublicKeyInfo
    )

def load_private_key(pem_data: bytes):
    """Загружает закрытый ключ из PEM-байт."""
    return serialization.load_pem_private_key(
        pem_data, password=None, backend=default_backend()
    )

```

```

def load_public_key(pem_data: bytes):
    """Загружает открытый ключ из PEM-байт."""
    return serialization.load_pem_public_key(pem_data,
        backend=default_backend())

def encrypt(public_key, data: bytes) -> dict:
    """
    Гибридное шифрование: генерирует AES-256 ключ, шифрует данные
    AES-CBC,
    затем шифрует AES-ключ с помощью RSA-OAEP.
    Возвращает словарь {"enc_key", "iv", "ct"} с
    base64-значениями.
    """
    aes_key = os.urandom(32)
    aes_iv = os.urandom(16)

    padder = PKCS7(128).padder()
    padded = padder.update(data) + padder.finalize()

    cipher = Cipher(algorithms.AES(aes_key), modes.CBC(aes_iv),
        backend=default_backend())
    ct = cipher.encryptor().update(padded) +
        cipher.encryptor().finalize()

    enc_key = public_key.encrypt(
        aes_key,
        rsa_padding.OAEP(
            mgf=rsa_padding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None
        )
    )
    return {
        "enc_key": base64.b64encode(enc_key).decode(),
        "iv": base64.b64encode(aes_iv).decode(),
        "ct": base64.b64encode(ct).decode(),
    }

def decrypt(private_key, payload: dict) -> bytes:
    # Гибридное дешифрование: расшифровывает AES-ключ с помощью
    RSA,
    затем дешифрует данные с помощью AES-CBC.

```

```

aes_key = private_key.decrypt(
    base64.b64decode(payload["enc_key"]),
    rsa_padding.OAEP(
        mgf=rsa_padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)
aes_iv = base64.b64decode(payload["iv"])
ct = base64.b64decode(payload["ct"])

cipher = Cipher(algorithms.AES(aes_key), modes.CBC(aes_iv),
backend=default_backend())
pt_padded = cipher.decryptor().update(ct) +
cipher.decryptor().finalize()

unpadder = PKCS7(128).unpadder()
return unpadder.update(pt_padded) + unpadder.finalize()

```

hashing.py

```

import hashlib
ALGORITHMS = ["MD5", "SHA-1", "SHA-256", "SHA-512", "SHA3-256",
"SHA3-512"]

_MAPPING = {
    "MD5":      hashlib.md5,
    "SHA-1":    hashlib.sha1,
    "SHA-256":  hashlib.sha256,
    "SHA-512":  hashlib.sha512,
    "SHA3-256": hashlib.sha3_256,
    "SHA3-512": hashlib.sha3_512,
}

def compute(algo: str, data: bytes) -> str:
    # Вычисляет хэш данных заданным алгоритмом. Возвращает
    # hex-строку.
    fn = _MAPPING.get(algo)
    if fn is None:

```

```
        raise ValueError(f"Неизвестный алгоритм: {algo}")
    return fn(data).hexdigest()

def compute_all(algorithms: list[str], data: bytes) -> dict[str, str]:
    #Вычисляет хэши сразу несколькими алгоритмами. Возвращает
    {algo: hex}.
    return {algo: compute(algo, data) for algo in algorithms}

def format_results(file_path: str, results: dict[str, str]) -> str:
    lines = [f"Файл: {file_path}\n"]
    for algo, h in results.items():
        lines.append(f"{algo:<12}: {h}")
    return "\n".join(lines)
```